Patrick Foulis
Lab 4
CS455 Computer Networks
20 September, 2026

Character Counts

Output:

```
'A'            chars= 1 bytes= 1 41
'é'            chars= 1 bytes= 2 c3 a9
'€'            chars= 1 bytes= 3 e2 82 ac
'🔌'            chars= 1 bytes= 4 f0 9f 94 8c
'naïve café'   chars=10 bytes=12 6e 61 c3 af 76 65 20 63 61 66 c3 a9
```

Characters can be up to 4 bytes so an appropriate buffer would have 32 * 4 bytes available, thus 128 bytes.

Mojibake

```
MacBook-Air-4 ~ % printf 'caf\xc3\xa9\n' | iconv -f latin1 -t utf-8
cafÃ©

MacBook-Air-4 ~ % printf 'caf\xe9\n' | iconv -f utf-8 -t utf-8
caficonv: unexpected end of file; the last character is incomplete.
```

In the first example, the valid UTF-8 string of hex bytes was sent and told to decode in Latin-1. Because these bytes are interpreted differently in the two charsets, the output was the correct but unexpected output of random symbols. The second example did the same thing but the other way around sending Latin-1 and receiving in UTF-8. This threw an error because the subsequent bytes in the stream did not follow the required UTF-8 bit pattern. Therefore an error was thrown.

Base64

```
MacBook-Air-4 ~ % head -c 3000 /dev/urandom > /tmp/blob.bin
base64 < /tmp/blob.bin | tr -d '\n' > /tmp/blob.b64
wc -c /tmp/blob.bin /tmp/blob.b64
    3000 /tmp/blob.bin
    4000 /tmp/blob.b64
    7000 total
```

The two byte counts were 3000 and 4000 bytes. The ration is an expansion of 4/3. WE accept this size overhead to safely transmit raw binary data over legacy, text-only communication channels that would otherwise corrupt, drop, or alter raw 8-bit binary bytes.

Compression

```
https://en.wikipedia.org/wiki/Computer_network
identity: 748020 bytes
compressed: 126651 bytes

> Accept-Encoding: deflate, gzip
< content-type: text/html; charset=UTF-8
< content-encoding: gzip
< vary: Accept-Encoding,X-Subdomain,Cookie,Authorization,User-Agent
```

The html was compressed by ~ 83%. And the Accept Encoding was the request header from the client. And the other three lines were the response header by the sender.

Handshake Ordering

TCP SYN Packet: 1
Client Hello Packet: 4
Server Hello Packet: 5

Two packets matched the filters: packet 4 and 5. The reason that none of the certificate information can be seen is because immediately after the handshake that was listed above, the encryption keys are created and distributed. Thus the certificates and other application layer items are encrypted and cannot be seen.

The ClientHello in the clear

SNI: em.wikipedia.org
Full APLN List: h2, http/1.1
Number of Cipher Suites: 49
(a) Version on outer TLS Record Layer: TLS 1.0 (0x0301)
(b) Version inside the Client Hello message: TLS 1.2 (0x0303)
(c) Highest entry in supported_versions extension: TLS 1.3

What an Observer Learns

(a) Which website was requested: Yes, this can be determined from the server_name extension (SNI) in the Client Hello message (en.wikipedia.org).

(b) Which page on that site: No, this cannot be determined because the specific HTTP request path is part of the encrypted HTTP payload sent inside Application Data records.

(c) Which application protocol will be used: Yes, determined from the selected_protocol field in the application_layer_protocol_negotiation (ALPN) extension inside the Server Hello message.

(d) The contents of the page: No, the page contents are completely encrypted within the Application Data records following the handshake.

Negotiated Parameters

Negotiated Protocol Version: TLSv1.3

Selected Cipher Suite: TLS_AES_128_GCM_SHA256

Selected ALPN Protocol: h2

Verification Result: Verification: OK

In the Wireshark capture we were unable to see the Selected Cipher Suite and the Verification. We were able to see the Negotiated Protocol version and the ALPN protocol in the Wireshark and this open ssl capture.

Certificate Chain

Subject: CN=*.wikipedia.org

Issuer: C=US, O=Let's Encrypt, CN=YE2

Chain Length: 4 (0,1,2,3)

For the Verification to be possible, the computer must already possess a pre-installed, trusted Root Certificate that anchors the top of the certificate validation chain.

Depreciated Version

```
MacBook-Air-4 ~ % openssl s_client -connect en.wikipedia.org:443
-servername en.wikipedia.org \
-tls1_1 </dev/null 2>&1 | grep -E 'no protocols|handshake has read'
80E102EF01000000:error:0A0000BF:SSL routines:tls_setup_handshake:no
protocols available:ssl/statem/statem_lib.c:155:
SSL handshake has read 0 bytes and written 7 bytes

MacBook-Air-4 ~ % openssl s_client -connect en.wikipedia.org:443
-servername en.wikipedia.org \
-min_protocol TLSv1.1 -max_protocol TLSv1.1 -cipher
'DEFAULT@SECLEVEL=0' \
</dev/null 2>&1 | grep -E 'alert|handshake has read'
```

```
80E102EF01000000:error:0A00042E:SSL routines:ssl3_read_bytes:tlsv1
alert protocol version:ssl/record/rec_layer_s3.c:918:SSL alert number
70
SSL handshake has read 7 bytes and written 130 bytes
```

In the first connection attempt, the refusal was issued locally by my own machine's openssl library. Proven by the SSL handshake has read output returning 0 bytes before any traffic reached the wire. Conversely, forcing the client to send the request resulted in a refusal from the remote Wikipedia server, evidenced by the output reading 7 bytes—representing a TLS Alert record returned across the network. That response contained TLS Alert number 70 (protocol_version), which was sent directly by the remote server to signal that it does not support or accept the deprecated TLS 1.1 protocol.

Session Resumption

```
MacBook-Air-4 ~ % { printf 'GET / HTTP/1.1\r\nHost:
en.wikipedia.org\r\nConnection: close\r\n\r\n'
sleep 2; } \
| openssl s_client -connect en.wikipedia.org:443 -servername
en.wikipedia.org \
-sess_out /tmp/lab4-sess.pem 2>/dev/null | grep -E '^(New|Reused)'
New, TLSv1.3, Cipher is TLS_AES_128_GCM_SHA256

MacBook-Air-4 ~ % { printf 'GET / HTTP/1.1\r\nHost:
en.wikipedia.org\r\nConnection: close\r\n\r\n'
sleep 2; } \
| openssl s_client -connect en.wikipedia.org:443 -servername
en.wikipedia.org \
-sess_in /tmp/lab4-sess.pem 2>/dev/null | grep -E '^(New|Reused)'
Reused, TLSv1.3, Cipher is TLS_AES_128_GCM_SHA256
```

Even though the second connection required a new TCP handshake at Layer 4, the TLS layer reused the cryptographic session. This remembered session state operates at the Session Layer (Layer 5) of the OSI model. This shows that even though Layer 5 in this day and age is obsolete, it never completely went away it was just wrapped into the HTTP protocols. This older version shows why it was given it’s own layer in the OSI model.

Six Specification Questions

1)
Code point U+20AC in 16-bit binary is 0010 0000 1010 1100. According to Section 3 of RFC 3629, it falls in the range 0000 0800 - 0000 FFFF (1110xxxx 10xxxxxx 10xxxxxx), which encodes to the 3-byte sequence 0xE2 0x82 0xAC.
2)
"*Implementations of the decoding algorithm above MUST protect against decoding invalid sequences… Decoding invalid sequences may have security consequences or cause other problems*."

Block over-long byte encodings to prevent attackers from bypassing security filters.If your system runs security checks before decoding the input, attackers can use alternative byte patterns to hide malicious code past the filter, which then gets decoded and executed safely inside the application. Decoding or rejecting over-long forms first closes this loophole.

3)
Clarified key updates and session ticket lifetime mechanics. Tightened rules regarding cipher suite negotiation and extension processing. Neither change alters what physically appears on the wire; they only clarify and refine implementation behavior and security edge cases.
4)
RFC 9846 updated master/slave and client/server-side authority terms to use inclusive, neutral language
5)
Phase 1: Key Exchange Phase

Phase 2: Server Parameters Phase

Phase 3: Authentication Phase

In the Wireshark capture The Client Hello and Server Hello packets belong to the Key Exchange Phase

6)
"If this extension is present, the server MUST select a TLS version from among those listed in the extension and ignore the legacy_version field in the ClientHello."

Modern TLS uses a clever trick so older network hardware won't break the connection: it pretends to be an older version like TLS 1.0 or 1.2 on the outside, while hiding its real request for TLS 1.3 inside the supported_versions extension. Older hardware lets the traffic pass right through because the outer version looks familiar, while modern servers check the extension and upgrade the connection to TLS 1.3 without any issues.

Reflection

I was surprised through our Wireshark exploration how much of the header data was unencrypted. I guess that there was not a lot there that when set up correctly was dangerous from a security standpoint. However prior to this class I would have assumed that the metadata from these connections would have been encrypted. And the supported versions extension was clever, and the type of intuitively that I enjoy learning about.